

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA0201

COURSE NAME: C Programming

COURSE OUTCOME COVERED

CO1: Construct structured C programs using constants, variables, data types, and preprocessor directives to address basic computational tasks. (BL2, BL3)

Assessment Tool Weightage: 20%

QUESTIONS: Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

I, Ashira Bridget Susan S (192524316), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Ashira Bridget Susan S

CO1 – EXPERIMENTS

Q1. Write a C program to:

- Display intermediate values using a macro DEBUG

Perform:

- Run with DEBUG = 1
- Run with DEBUG = 0

Record:

- Output differences

Conclude:

- How macros control program behavior

When debug = 1:

main.c	Output
<pre>1 #include <stdio.h> 2 #define DEBUG 1 3 4 int main() 5 { 6 int a = 10, b = 20, sum; 7 8 sum = a + b; 9 10 #if DEBUG 11 printf("a = %d\nb = %d\n", a, b); 12 #endif 13 14 printf("Sum = %d", sum); 15 16 return 0; 17 }</pre>	<pre>a = 10 b = 20 Sum = 30 === Code Execution Successful ===</pre>

When debug = 2:

main.c	Output
<pre>1 #include <stdio.h> 2 #define DEBUG 0 3 4 int main() 5 { 6 int a = 10, b = 20, sum; 7 8 sum = a + b; 9 10 #if DEBUG 11 printf("a = %d\nb = %d\n", a, b); 12 #endif 13 14 printf("Sum = %d", sum); 15 16 return 0; 17 }</pre>	<pre>Sum = 30 === Code Execution Successful ===</pre>

Q2. Data Type Experiment

Write a program to calculate average of 3 numbers using:

- int
- float

Perform:

- Run both versions

Observe:

- Difference in output

Conclude:

- Effect of data types on accuracy

main.c	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int a=10,b=20,c=30; 6 float x=10,y=20,z=30; 7 8 printf("Integer Average = %d\n", (a+b+c)/3); 9 printf("Float Average = %.2f\n", (x+y+z)/3); 10 11 return 0; 12 }</pre>	<pre>Integer Average = 20 Float Average = 20.00 === Code Execution Successful ===</pre>

Observation: Float preserves decimal values.

Conclusion: Float provides better accuracy than int.

Q3. Constant vs Variable Experiment

Write two versions of a program to compute the circumference of a circle:

- One using `#define` for constants
- One using variables

Perform:

- Change values and run

Observe:

- Which is easier to modify

Conclude:

- Advantage of macros

Circumference using Macro:

<pre>main.c 1 #include <stdio.h> 2 #define PI 3.14 3 4 int main() 5 { 6 float r=5; 7 printf("Circumference = %.2f",2*PI*r); 8 return 0; 9 }</pre>	Run Output Circumference = 31.40 === Code Execution Successful ===
---	--

Circumference using variable:

<pre>main.c 1 #include <stdio.h> 2 3 int main() 4 { 5 float pi=3.14,r=5; 6 printf("Circumference = %.2f",2*pi*r); 7 return 0; 8 }</pre>	Run Output Circumference = 31.40 === Code Execution Successful ===
---	--

Observation: Changing PI using macros is easier.

Conclusion: Macros improve readability and maintenance.

Q4. Type Casting Experiment

Write a program to compute average:

- Without type casting
- With type casting

Observe:

- Output difference

Conclude:

- Why type casting is required

main.c	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int a=10,b=20,c=25; 6 7 printf("Without Casting = %d\n", (a+b+c)/3); 8 printf("With Casting = %.2f", (float)(a+b+c)/3); 9 10 return 0; 11 }</pre>	<pre>Without Casting = 18 With Casting = 18.33 === Code Execution Successful ===</pre>

Conclusion: Type casting prevents loss of precision.

Q5. Input Validation Experiment

Write a program to:

- Accept input within a defined range using constants

Perform:

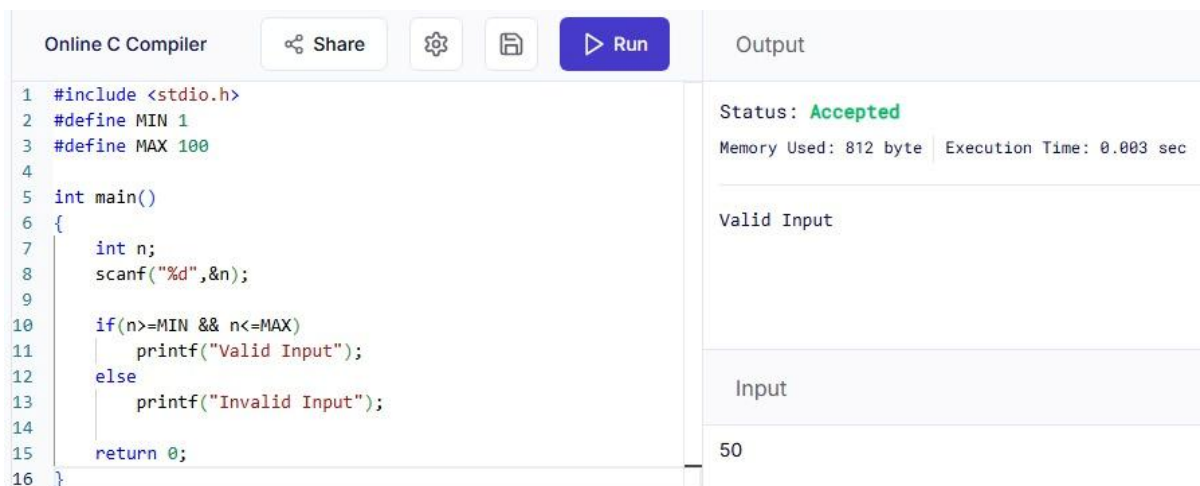
- Test with valid and invalid inputs

Observe:

- Program response

Conclude:

- Importance of validation



The screenshot displays an online C compiler interface. On the left, a code editor shows a C program that defines constants MIN (1) and MAX (100), then checks if user input is within this range. The right side shows the execution status as 'Accepted' with memory and time usage details. Below this, the program's output is shown as 'Valid Input' for the input value 50.

```
1 #include <stdio.h>
2 #define MIN 1
3 #define MAX 100
4
5 int main()
6 {
7     int n;
8     scanf("%d",&n);
9
10    if(n>=MIN && n<=MAX)
11        printf("Valid Input");
12    else
13        printf("Invalid Input");
14
15    return 0;
16 }
```

Online C Compiler Share Settings Save Run

Status: **Accepted**
Memory Used: 812 byte Execution Time: 0.003 sec

Valid Input

Input

50

Conclusion: Input validation avoids incorrect data.

Q6. Macro-Based Array Size

Write a program where array size is defined using macro

Perform:

- Change size and re-run

Observe:

- Output behavior

Conclude:

- Benefit of compile-time configuration

Online C Compiler	Share	Settings	Save	Run	Output
<pre>1 #include <stdio.h> 2 #define SIZE 5 3 4 int main() 5 { 6 int a[SIZE],i; 7 8 for(i=0;i<SIZE;i++) 9 scanf("%d",&a[i]); 10 11 for(i=0;i<SIZE;i++) 12 printf("%d ",a[i]); 13 14 return 0; 15 } 16</pre>					Status: Accepted Memory Used: 728 byte Execution Time: 0.003 sec 10 20 30 40 50 Input 10 20 30 40 50

Conclusion: Compile-time configuration becomes easier.

Q7. Feature Enable/Disable Experiment

Write a program to read amount to be paid with

- Optional discount (if the amount is greater than 1000, give 5% discount) using macro

Perform:

- Enable and disable feature

Observe:

- Change in total value

Conclude:

- Flexibility of preprocessor directives

The screenshot shows an online C compiler interface. On the left, the C code is displayed with line numbers 1 through 18. The code includes `<stdio.h>`, defines a macro `DISCOUNT 1`, and contains a `main` function. Inside `main`, a `float amount;` is declared, and `scanf("%f",&amount);` is used to read input. An `if` statement checks `if(amount>1000)` and, if true, applies a 5% discount: `amount=amount-(amount*0.05);`. The code then prints the amount with two decimal places using `printf("Amount = %.2f",amount);` and returns 0. On the right, the 'Output' section shows the status as 'Accepted', memory usage of 736 bytes, and execution time of 0.003 seconds. Below this, the output of the program is shown as 'Amount = 1900.00'. At the bottom, the 'Input' section shows the value '2000'.

```
1 #include <stdio.h>
2 #define DISCOUNT 1
3
4 int main()
5 {
6     float amount;
7     scanf("%f",&amount);
8
9     #if DISCOUNT
10         if(amount>1000)
11             amount=amount-(amount*0.05);
12     #endif
13
14     printf("Amount = %.2f",amount);
15
16     return 0;
17 }
18
```

Output

Status: **Accepted**
Memory Used: 736 byte | Execution Time: 0.003 sec

Amount = 1900.00

Input

2000

Conclusion: Macros and preprocessor directives allow program features to be enabled or disabled without modifying the core logic. This makes the code more flexible, reusable, and easier to maintain.

Q8. Precision Experiment

Write a program to calculate area of circle

Perform:

- Use float and double

Observe:

- Precision difference

Conclude:

- Best data type choice

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 float rf=5.5; 6 double rd=5.5; 7 8 printf("Float Area = %.2f\n",3.14*rf*rf); 9 printf("Double Area = %.10lf",3.1415926535*rd*rd); 10 11 return 0; 12 }</pre>		<pre>Float Area = 94.98 Double Area = 95.0331777684 === Code Execution Successful ===</pre>

Conclusion: The double data type provides greater precision than float and is more suitable for scientific and mathematical calculations where accuracy is important.

Q9. Overflow Experiment

Write a program to:

- Store large values in different data types

Perform:

- Test with large inputs

Observe:

- Overflow behavior

Conclude:

- Limitations of data types

main.c	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 char a=130; 6 int b=40000; 7 8 printf("char = %d\n",a); 9 printf("int = %d",b); 10 11 return 0; 12 }</pre>	<pre>char = -126 int = 40000 === Code Execution Successful ===</pre>

Conclusion: Every data type has a limited storage range. Choosing an inappropriate data type for large values can lead to overflow and incorrect results, highlighting the importance of proper data type selection.

Q10. Multi-Case Testing

Write a program to:

- Convert temperature (Kelvin to Celcius, Kelvin to Farenheit)

Perform:

- Test with 3 different inputs

Observe:

- Output correctness

Conclude:

- Program reliability

Online C Compiler	Share	Settings	Save	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 float k; 6 7 scanf("%f",&k); 8 9 printf("Celsius = %.2f\n",k-273.15); 10 printf("Fahrenheit = %.2f", (k-273.15)*9/5+32); 11 12 return 0; 13 } 14</pre>					Status: Accepted Memory Used: 720 byte Execution Time: 0.004 sec Celsius = 26.85 Fahrenheit = 80.33
					Input
					300

Conclusion: Testing a program with multiple inputs helps verify its correctness and reliability. It ensures that the program handles different cases accurately and produces consistent results.

Q11. Replace Hardcoding

Write a program to swap two values. Hard code the values.

Modify:

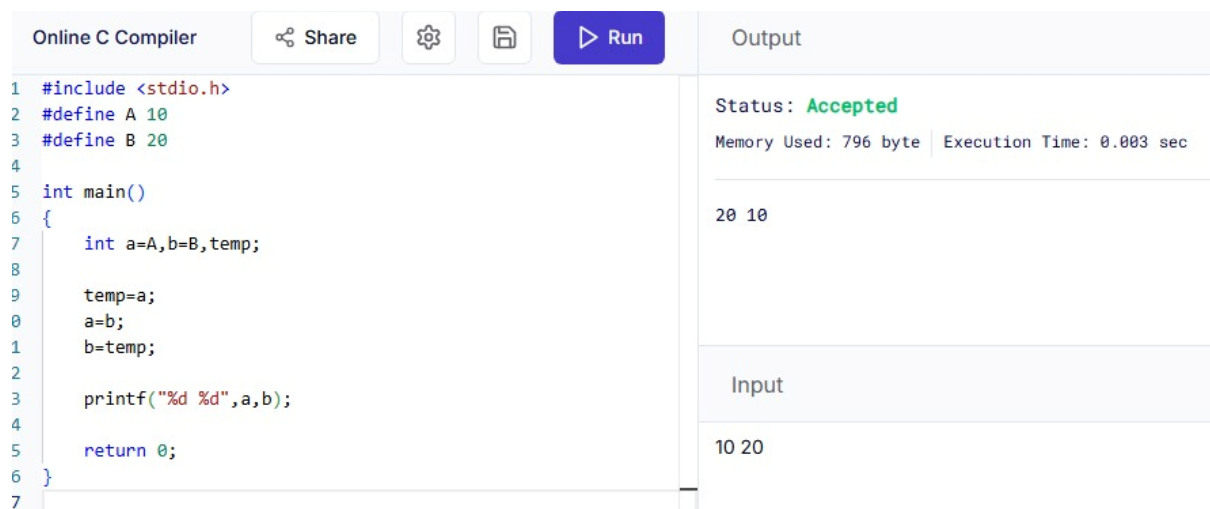
- Replace with macros

Observe:

- Code readability

Conclude:

- Importance of symbolic constants



The screenshot shows an online C compiler interface. On the left, the code editor contains the following C program:

```
1 #include <stdio.h>
2 #define A 10
3 #define B 20
4
5 int main()
6 {
7     int a=A,b=B,temp;
8
9     temp=a;
10    a=b;
11    b=temp;
12
13    printf("%d %d",a,b);
14
15    return 0;
16 }
```

On the right, the output panel shows the status "Accepted" in green, with "Memory Used: 796 byte" and "Execution Time: 0.003 sec". Below this, the output of the program is displayed as "20 10". At the bottom of the output panel, there is an "Input" section showing "10 20".

Conclusion: Replacing hardcoded values with symbolic constants (macros) improves code readability, simplifies modifications, and reduces the chances of errors when updating values in a program.

Q12. Control Structure Experiment

Write a program to read three marks and calculate the grade based on the following constraints:

- if the student fails in any one subject his grade is displayed as 'F'.
- if the average is > 90 , display 'S'
- if the average is > 80 , display 'A'
- if the average is > 70 , display 'B'

Perform:

- Test boundary values

Observe:

- Output correctness

Conclude:

- Role of conditions

Online C Compiler	Share	Settings	Save	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int a,b,c; 6 float avg; 7 8 scanf("%d%d%d",&a,&b,&c); 9 10 if(a<50 b<50 c<50) 11 printf("Grade = F"); 12 else 13 { 14 avg=(a+b+c)/3.0; 15 16 if(avg>90) 17 printf("Grade = S"); 18 else if(avg>80) 19 printf("Grade = A"); 20 else if(avg>70) 21 printf("Grade = B"); 22 else 23 printf("Grade = C"); 24 }</pre>					Status: Accepted Memory Used: 720 byte Execution Time: 0.004 sec Grade = F Input 80 35 90

Conclusion: Conditions ensure proper grading.

Q13. Step-by-Step Output Tracking

Write a program to calculate the electricity consumption.

Add:

- Print statements using macro

Observe:

- Intermediate results

Conclude:

- Debugging benefits

Online C Compiler	Share	Settings	Save	Run	Output
<pre>1 #include <stdio.h> 2 #define DEBUG 1 3 4 int main() 5 { 6 int units=200; 7 float bill=units*5; 8 9 #if DEBUG 10 printf("Units = %d\n",units); 11 #endif 12 13 printf("Bill = %.2f",bill); 14 15 return 0; 16 }</pre>					Status: Accepted Memory Used: 720 byte Execution Time: 0.003 sec Units = 200 Bill = 1000.00 Input 200

Conclusion: Debug statements help in troubleshooting.

Q14. Minimal Memory Design

Write a program that reads cost of an item, number of items bought and calculate the total amount to be paid:

Apply:

- smallest suitable data types

Perform:

- Compare with normal version

Observe:

- Efficiency

Conclude:

- Importance of data selection

Online C Compiler	Share	Settings	Save	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 short cost,items; 6 int total; 7 8 scanf("%hd%hd",&cost,&items); 9 10 total=cost*items; 11 12 printf("Total = %d",total); 13 14 return 0; 15 }</pre>					Status: Accepted Memory Used: 720 byte Execution Time: 0.003 sec Total = 250 Input 50 5

Conclusion: Choosing suitable data types improves memory efficiency.

Q15. Program Modification Task

Write a basic program to find the celsius using Farenheit

Modify:

- Add macros
- Change data types

Observe:

- Output changes

Conclude:

- Impact of design choices

Online C Compiler	Share	Settings	Save	Run	Output
<pre>1 #include <stdio.h> 2 #define VALUE 32 3 4 int main() 5 { 6 float f=100,c; 7 8 c=(f-VALUE)*5/9; 9 10 printf("Celsius = %.2f",c); 11 12 return 0; 13 }</pre>					Status: Accepted Memory Used: 716 byte Execution Time: 0.003 sec Celsius = 37.78
					Input
					100

Observation: Changing macros or data types changes the output precision and flexibility.

Conclusion: Good program design improves readability, maintainability, and accuracy.